

KUIS
PEMROGRAMAN BERORIENTASI OBJEK
POLITEKNIK NEGERI MALANG
TAHUN 2026-2027



Oleh:
Nico Lee Nathanael Handoko
(254107060005)

PROGRAM STUDI
D-IV SISTEM INFORMASI BISNIS
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG
SEPTEMBER 2026



Jurusan Teknologi Informasi, Politeknik Negeri Malang
Program Studi Sistem Informasi Bisnis
Pemrograman Berorientasi Objek (Praktikum)
Kuis 1

Tujuan Kuis

Setelah menyelesaikan kuis ini, siswa diharapkan dapat:

1. Mampu menjelaskan konsep dasar Pemrograman Berorientasi Objek (OOP).
2. Implementasikan kelas dan objek menggunakan Java.
3. Terapkan enkapsulasi menggunakan atribut privat, konstruktor, getter, dan setter.
4. Identifikasi dan terapkan hubungan yang tepat antar kelas.
5. Membuat program Java sederhana berdasarkan studi kasus tertentu menggunakan prinsip OOP.

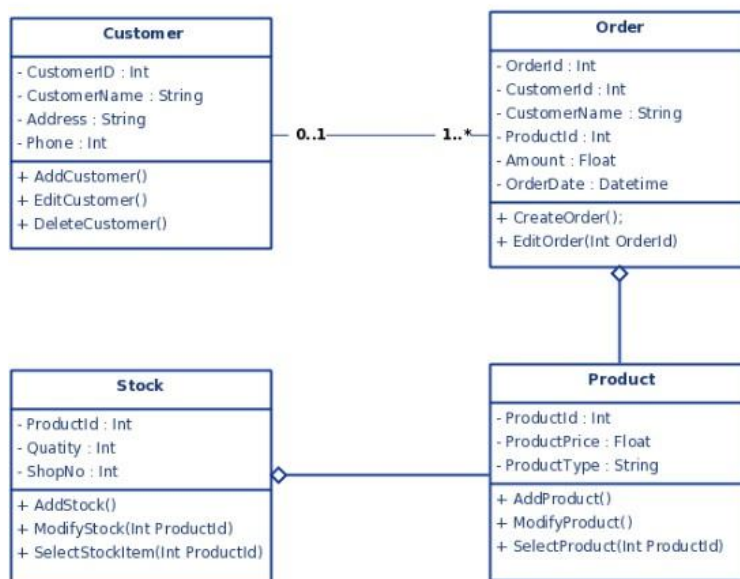
Aturan dan Persyaratan Kuis

1. Kuis ini harus diselesaikan secara individual.
2. Kirim satu file PDF yang berisi :
 - o Kode program lengkap.
 - o Tangkapan layar yang menunjukkan output program.
3. Menyalin atau menggunakan kode dari teman sekelas atau alat AI sangat dilarang.
4. Selesaikan kuis dengan keterampilan dan pemahaman Anda sendiri. Percaya diri, percayalah pada kemampuan Anda, dan lakukan yang terbaik.

Kuis:

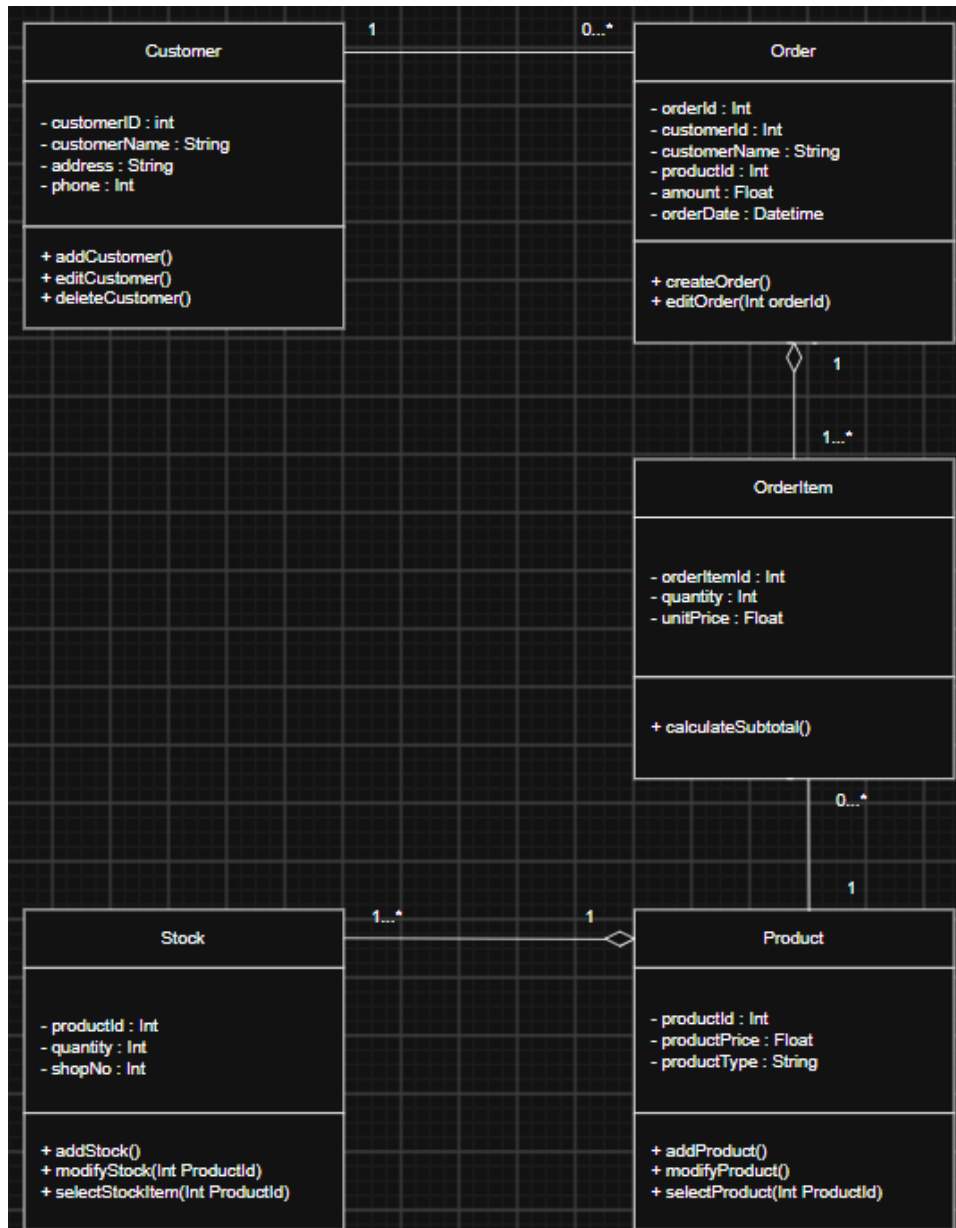
1. Periksa diagram kelas berikut. Koreksi diagram kelas sesuai dengan notasi UML yang tepat dan justifikasi koreksi yang Anda buat. Setelah merevisi diagram, jelaskan hubungan antara kelas dan notasi UML yang digunakan. Terakhir, implementasikan diagram kelas yang telah dikoreksi sebagai program Java.

Class Diagram for Order Processing System



Link Github: <https://github.com/Nico-LeeNH/PBO/tree/main/kuis>

Jawab:



Di class diagram ini, saya tambahkan class OrderItem karena adanya hubungan many to many antara product dengan Order.

Relasi hubungan antar kelas:

- **Customer – Order** menggunakan notasi **Asosiasi biasa**(garis polos), **1 – 0..*** dimana **satu customer** bisa **punya 0 sampai banyak order**.
- **Order – OrderItem** menggunakan notasi **Komposisi**(diamond solid di sisi order), **1 - 1..*** dimana **satu order** terdiri dari **1 atau lebih item**.
- **OrderItem – Product** menggunakan notasi **Asosiasi biasa**, **0..* - 1** dimana **satu produk** dapat dipesan lewat **banyak baris order item**, tapi **tiap order item** pasti punya **satu produk**.
- **Stock – Product** menggunakan notasi **Agregasi**(diamond kosong di sisi product), **1..* - 1** dimana **satu product** bisa punya **satu hingga banyak stock** dan **satu stock** hanya punya **satu product**.

Code implemetasi dari class diagram:

```
1  import java.util.ArrayList;
2  import java.util.Date;
3  import java.util.List;
4
5  // relasi 1 Customer -> banyak Order
6  class Customer {
7      private int customerId;
8      private String customerName;
9      private String address;
10     private String phone;
11     private List<Order> orders = new ArrayList<>();
12
13     public Customer(int customerId, String customerName, String address, String phone) {
14         this.customerId = customerId;
15         this.customerName = customerName;
16         this.address = address;
17         this.phone = phone;
18     }
19
20     public void addCustomer() {
21         System.out.println("Customer baru ditambahkan: " + customerName);
22     }
23
24     public void editCustomer() {
25         System.out.println("Data customer " + customerName + " diperbarui.");
26     }
27
28     public void deleteCustomer() {
29         System.out.println("Customer " + customerName + " dihapus.");
30     }
31
32     // dipanggil oleh Order saat dibuat, menjaga konsistensi relasi dua arah
33     void tambahOrder(Order order) {
34         orders.add(order);
35     }
36
37     public int getCustomerId() {
38         return customerId;
39     }
40
41     public String getCustomerName() {
42         return customerName;
43     }
44
45     public List<Order> getOrders() {
46         return orders;
47     }
48 }
```

```
1 // relasi agregasi: 1 Product -> banyak Stock
2 class Product {
3     private int productId;
4     private float productPrice;
5     private String productType;
6     private List<Stock> stokDiCabang = new ArrayList<>();
7
8     public Product(int productId, float productPrice, String productType) {
9         this.productId = productId;
10        this.productPrice = productPrice;
11        this.productType = productType;
12    }
13
14    public void addProduct() {
15        System.out.println("Product baru ditambahkan, id = " + productId);
16    }
17
18    public void modifyProduct() {
19        System.out.println("Product id=" + productId + " diperbarui.");
20    }
21
22    public Product selectProduct(int productId) {
23        System.out.println("Mencari product id=" + productId);
24        return this;
25    }
26
27    void tambahStock(Stock stock) {
28        stokDiCabang.add(stock);
29    }
30
31    public int getProductId() {
32        return productId;
33    }
34
35    public float getProductPrice() {
36        return productPrice;
37    }
38
39    public String getProductType() {
40        return productType;
41    }
42 }
```

```

1 // Product mengagregasi Stock
2 class Stock {
3     private int productId;
4     private int quantity;
5     private int shopNo;
6
7     public Stock(Product product, int quantity, int shopNo) {
8         this.productId = product.getProductId();
9         this.quantity = quantity;
10        this.shopNo = shopNo;
11        product.tambahStock(this);
12    }
13
14    public void addStock() {
15        System.out.println("Stock baru ditambahkan untuk product id = " + productId);
16    }
17
18    public void modifyStock(int productId) {
19        System.out.println("Stock product id = " + productId + " diperbarui");
20    }
21
22    public Stock selectStockItem(int productId) {
23        System.out.println("Mencari stock product id = " + productId);
24        return this;
25    }
26
27    public int getQuantity() {
28        return quantity;
29    }
30
31    public int getShopNo() {
32        return shopNo;
33    }
34 }

```

```

1 // hasil many-to-many Order dan Product
2 class OrderItem {
3     private Product product;
4     private int quantity;
5     private float unitPrice;
6
7     public OrderItem(Product product, int quantity) {
8         this.product = product;
9         this.quantity = quantity;
10        this.unitPrice = product.getProductPrice();
11    }
12
13    public float hitungSubtotal() {
14        return unitPrice * quantity;
15    }
16
17    public Product getProduct() {
18        return product;
19    }
20
21    public int getQuantity() {
22        return quantity;
23    }
24 }

```

```

1  class Order {
2      private int orderId;
3      private Date orderDate;
4      private Customer customer;
5      // relasi komposisi: 1 Order -> banyak OrderItem
6      private List<OrderItem> items = new ArrayList<>();
7
8      public Order(int orderId, Customer customer) {
9          this.orderId = orderId;
10         this.orderDate = new Date();
11         this.customer = customer;
12         customer.tambahOrder(this);
13     }
14
15     public void createOrder() {
16         System.out.println("Order #" + orderId + " dibuat untuk " + customer.getCustomerName());
17     }
18
19     public void editOrder(int orderId) {
20         System.out.println("Order " + orderId + " diperbarui");
21     }
22
23     public void tambahItem(Product product, int quantity) {
24         items.add(new OrderItem(product, quantity));
25     }
26
27     public float hitungTotal() {
28         float total = 0;
29         for (OrderItem item : items) {
30             total += item.hitungSubtotal();
31         }
32         return total;
33     }
34
35     public int getOrderId() {
36         return orderId;
37     }
38
39     public List<OrderItem> getItems() {
40         return items;
41     }
42 }

```

```

1  public class OrderProcessingSystem {
2      public static void main(String[] args) {
3          Customer c1 = new Customer(1, "Adi Suloso", "Jl. Kayutangan No. 10", "08789564322");
4          c1.addCustomer();
5
6          Product p1 = new Product(001, 15000, "Makanan");
7          Product p2 = new Product(002, 25000, "Minuman");
8          p1.addProduct();
9          p2.addProduct();
10
11         new Stock(p1, 50, 1);
12         new Stock(p1, 30, 2); // produk yang sama, stok di cabang berbeda (agregasi)
13         new Stock(p2, 20, 1);
14
15         Order order1 = new Order(1011, c1);
16         order1.createOrder();
17         order1.tambahItem(p1, 3);
18         order1.tambahItem(p2, 3);
19
20         System.out.println("\n=== Ringkasan Order ===");
21         System.out.println("Order " + order1.getOrderId() + " milik " + c1.getCustomerName());
22         for (OrderItem item : order1.getItems()) {
23             System.out.println("- Product id " + item.getProduct().getProductId()
24                 + " x" + item.getQuantity()
25                 + " = Rp" + item.hitungSubtotal());
26         }
27         System.out.println("Total: Rp" + order1.hitungTotal());
28         System.out.println("Jumlah order: " + c1.getOrders().size());
29     }
30 }

```

Output:

```
PS D:\polinema\PBO\kuis> java OrderProcessingSystem.java
Customer baru ditambahkan: Adi Suloso
Product baru ditambahkan, id = 1
Product baru ditambahkan, id = 2
Order #1011 dibuat untuk Adi Suloso

=== Ringkasan Order ===
Order 1011 milik Adi Suloso
- Product id 1 x3 = Rp45000.0
- Product id 2 x3 = Rp75000.0
Total: Rp120000.0
Jumlah order: 1
```

2. Sebuah bengkel otomotif bernama "**Bengkel Maju**" ingin mengembangkan sistem sederhana untuk mengelola data kendaraan pelanggan bagi kendaraan yang dibawa untuk diperbaiki. Sistem berbasis Java harus mampu merekam informasi kendaraan (mobil dan sepeda motor), menampilkan informasi layanan, dan menghitung perkiraan biaya berdasarkan jenis kendaraan dan jenis layanan. Sistem ini harus menerapkan prinsip pemrograman berorientasi objek:

1. *Kelas pelanggan* dengan atribut `nama` dan `Nomor telepon`.
2. *Kelas kendaraan* dengan atribut `platNomor`, `merek`, `model`, dan `tipe kendaraan`.
3. *Kelas layanan* dengan atribut `serviceName` dan `servicePrice`.
4. Tambahkan satu *kelas Karyawan* dengan atribut dan metode yang sesuai berdasarkan persyaratan sistem.
5. Terapkan *enkapsulasi* pada semua atribut menggunakan *atribut privat*, konstruktor, getter, dan setter.
6. Bangun hubungan antar kelas sehingga satu pelanggan dapat memiliki beberapa kendaraan, sementara setiap kendaraan hanya dimiliki oleh satu pelanggan. Definisikan hubungan yang sesuai yang melibatkan *kelas Layanan* dan *Karyawan*.
7. Buat program *Main* untuk menginstansiasi objek dan menampilkan semua informasi layanan.
8. Hitung total biaya layanan sesuai dengan aturan berikut:
 - Biaya layanan → mobil + Rp50.000 (biaya layanan)
 - Biaya layanan → sepeda motor + Rp20.000 (biaya layanan)

Output:

Tampilkan nama pelanggan, informasi kendaraan (termasuk beberapa contoh 2 mobil dan 2 sepeda motor), layanan terpilih, dan total perkiraan biaya.

Jawaban no2 ada di halaman selanjutnya..

Code:

```
1  import java.util.ArrayList;
2  import java.util.List;
3
4  class Pelanggan {
5      private String nama;
6      private String nomorTelepon;
7      private List<Kendaraan> daftarKendaraan = new ArrayList<>();
8
9      public Pelanggan(String nama, String nomorTelepon) {
10         this.nama = nama;
11         this.nomorTelepon = nomorTelepon;
12     }
13
14     void tambahKendaraan(Kendaraan kendaraan) {
15         daftarKendaraan.add(kendaraan);
16     }
17
18     public String getNama() { return nama; }
19     public String getNomorTelepon() { return nomorTelepon; }
20     public List<Kendaraan> getDaftarKendaraan() { return daftarKendaraan; }
21 }
22
23 abstract class Kendaraan {
24     private String platNomor;
25     private String merek;
26     private String model;
27     private String tipeKendaraan;
28     private Pelanggan pemilik;
29
30     public Kendaraan(String platNomor, String merek, String model, String tipeKendaraan, Pelanggan pemilik) {
31         this.platNomor = platNomor;
32         this.merek = merek;
33         this.model = model;
34         this.tipeKendaraan = tipeKendaraan;
35         this.pemilik = pemilik;
36         pemilik.tambahKendaraan(this);
37     }
38
39     // method abstrak: setiap jenis kendaraan punya biaya tambahan layanan yang berbeda
40     public abstract double getBiayaTambahan();
41
42     public String getPlatNomor() { return platNomor; }
43     public String getMerek() { return merek; }
44     public String getModel() { return model; }
45     public String getTipeKendaraan() { return tipeKendaraan; }
46     public Pelanggan getPemilik() { return pemilik; }
47
48     public String getInfo() {
49         return tipeKendaraan + " " + merek + " " + model + " (" + platNomor + ")";
50     }
51 }
```



```
1 // method abstrak: setiap jenis kendaraan punya biaya tambahan layanan yang berbeda
2 public abstract double getBiayaTambahan();
3
4 public String getPlatNomor() {
5     return platNomor;
6 }
7
8 public String getMerek() {
9     return merek;
10 }
11
12 public String getModel() {
13     return model;
14 }
15
16 public String getTipeKendaraan() {
17     return tipeKendaraan;
18 }
19
20 public Pelanggan getPemilik() {
21     return pemilik;
22 }
23
24 public String getInfo() {
25     return tipeKendaraan + " " + merek + " " + model + " (" + platNomor + ")";
26 }
27 }
```



```
1 class Mobil extends Kendaraan {
2     public Mobil(String platNomor, String merek, String model, Pelanggan pemilik) {
3         super(platNomor, merek, model, "Mobil", pemilik);
4     }
5
6     @Override
7     public double getBiayaTambahan() {
8         return 50000;
9     }
10 }
```



```
1 class SepedaMotor extends Kendaraan {
2     public SepedaMotor(String platNomor, String merek, String model, Pelanggan pemilik) {
3         super(platNomor, merek, model, "Sepeda Motor", pemilik);
4     }
5
6     @Override
7     public double getBiayaTambahan() {
8         return 20000;
9     }
10 }
11
```

```

12 class Layanan {
13     private String serviceName;
14     private double servicePrice;
15
16     public Layanan(String serviceName, double servicePrice) {
17         this.serviceName = serviceName;
18         this.servicePrice = servicePrice;
19     }
20
21     public String getServiceName() {
22         return serviceName;
23     }
24
25     public double getServicePrice() {
26         return servicePrice;
27     }
28 }
29
30 class Karyawan {
31     private String idKaryawan;
32     private String namaKaryawan;
33     private String posisi;
34     private List<Layanan> layananDitangani = new ArrayList<>();
35
36     public Karyawan(String idKaryawan, String namaKaryawan, String posisi) {
37         this.idKaryawan = idKaryawan;
38         this.namaKaryawan = namaKaryawan;
39         this.posisi = posisi;
40     }
41
42     // Karyawan mengerjakan sebuah layanan untuk kendaraan tertentu
43     public double kerjakanLayanan(Layanan layanan, Kendaraan kendaraan) {
44         layananDitangani.add(layanan);
45         double totalBiaya = layanan.getServicePrice() + kendaraan.getBiayaTambahan();
46         System.out.println(namaKaryawan + " (" + posisi + ") mengerjakan " + layanan.getServiceName()
47             + " untuk " + kendaraan.getInfo()
48             + " -> Rp" + totalBiaya);
49         return totalBiaya;
50     }
51
52     public String getNamaKaryawan() {
53         return namaKaryawan;
54     }
55
56     public List<Layanan> getLayananDitangani() {
57         return layananDitangani;
58     }
59 }

```

```

1 public class BengkelMaju {
2     public static void main(String[] args) {
3         Pelanggan p1 = new Pelanggan("Andi Chandra", "081211112222");
4         Pelanggan p2 = new Pelanggan("Rachma Nur C", "081333334444");
5
6         Mobil mobil1 = new Mobil("N 1234 ABC", "Toyota", "Avanza", p1);
7         Mobil mobil2 = new Mobil("N 5678 DEF", "Honda", "Brio", p2);
8         SepedaMotor motor1 = new SepedaMotor("L 1233 QR", "Yamaha", "NMAX", p1);
9         SepedaMotor motor2 = new SepedaMotor("B 4456 RQ", "Honda", "Beat", p2);
10
11         Layanan gantiOli = new Layanan("Ganti Oli", 100000);
12         Layanan servisRem = new Layanan("Servis Rem", 150000);
13         Layanan tuneUp = new Layanan("Tune Up", 200000);
14
15         Karyawan mekanik1 = new Karyawan("K01", "Joko", "Mekanik Senior");
16         Karyawan mekanik2 = new Karyawan("K02", "Asep", "Mekanik");
17
18         System.out.println("=== Proses Servis ===");
19         double totalMobil1 = mekanik1.kerjakanLayanan(gantiOli, mobil1);
20         double totalMobil2 = mekanik2.kerjakanLayanan(tuneUp, mobil2);
21         double totalMotor1 = mekanik1.kerjakanLayanan(servisRem, motor1);
22         double totalMotor2 = mekanik2.kerjakanLayanan(gantiOli, motor2);
23
24         System.out.println("\n=== Ringkasan Layanan perPelanggan ===");
25         cetakRingkasan(p1);
26         cetakRingkasan(p2);
27
28         double Total = totalMobil1 + totalMobil2 + totalMotor1 + totalMotor2;
29         System.out.println("Total perkiraan biaya seluruh layanan: Rp" + Total);
30     }
31
32     private static void cetakRingkasan(Pelanggan pelanggan) {
33         System.out.println("Pelanggan: " + pelanggan.getNama() + " (" + pelanggan.getNomorTelepon() + ")");
34         for (Kendaraan k : pelanggan.getDaftarKendaraan()) {
35             System.out.println("    - " + k.getInfo());
36         }
37     }
38 }

```

Output:

```

PS D:\polinema\PBO\kuis> java BengkelMaju.java
=== Proses Servis ===
Joko (Mekanik Senior) mengerjakan Ganti Oli untuk Mobil Toyota Avanza (N 1234 ABC) -> Rp150000.0
Asep (Mekanik) mengerjakan Tune Up untuk Mobil Honda Brio (N 5678 DEF) -> Rp250000.0
Joko (Mekanik Senior) mengerjakan Servis Rem untuk Sepeda Motor Yamaha NMAX (L 1233 QR) -> Rp170000.0
Asep (Mekanik) mengerjakan Ganti Oli untuk Sepeda Motor Honda Beat (B 4456 RQ) -> Rp120000.0

=== Ringkasan Layanan perPelanggan ===
Pelanggan: Andi Chandra (081211112222)
    - Mobil Toyota Avanza (N 1234 ABC)
    - Sepeda Motor Yamaha NMAX (L 1233 QR)
Pelanggan: Rachma Nur C (081333334444)
    - Mobil Honda Brio (N 5678 DEF)
    - Sepeda Motor Honda Beat (B 4456 RQ)
Total perkiraan biaya seluruh layanan: Rp690000.0

```

---- Semoga beruntung ----